

교과목 3 : 초거대언어모델(LLM)

단원 1 : 자연어 데이터 준비

DAY1. 자연어 처리 절차와 형태소 분석

목 차

1. 자연어 데이터의 특징		3
2. 텍스트 데이터 전처리		11
3. 형태소 분석		39
4. 워드 클라우드 시각화		51

1. 자연어 데이터의 특징

정형 데이터의 개념

1. 정형 데이터 정의

1. 표 형식으로 구조화된 데이터
2. 예: 발주 목록, 직원 정보(아이디/이름/나이/직업)

2. 정형 데이터의 특징

1. 필드 값이 명확함
2. 숫자/문자 구분 가능
 - ① 전화번호: 계산 대상 아님
 - ② 나이: 계산·비교 가능

3. 정형 데이터 처리 도구

1. Excel
2. Python
3. SQL(데이터베이스)

정형 데이터(Structured Data)는 일정한 형식과 구조를 가진 데이터이다.

주로 행(Row)과 열(Column)로 구성된 표 형태로 저장되며, 데이터의 의미가 명확하게 정의되어 있다.

예를 들어 직원 정보에서 **아이디, 이름, 나이, 직업**과 같은 항목은 각각 정해진 위치에 저장된다.

정형 데이터의 특징은 **숫자와 문자를 구분하여 저장하고 처리할 수 있다는 점**이다. 예를 들어 전화번호는 숫자로 보이지만 계산 대상이 아니고, 나이는 계산과 비교가 가능한 숫자 데이터이다.

정형 데이터는 주로 **Excel, Python, SQL(데이터베이스)** 등을 사용하여 저장, 조회, 분석 및 관리한다.

비정형 데이터의 개념

1. 비정형 데이터 개념

1. 표 형태로 구조화하기 어려운 데이터
2. 형태가 다양하고 표현 방식이 통일되어 있지 않음

2. 비정형 데이터의 주요 유형

1) 텍스트 기반

- ① 이메일
- ② SNS 글(인스타그램, 틱톡 등)
- ③ 게시물, 뉴스 기사, 리뷰

2) 이미지 기반

- ① 사진
- ② 그림/도형 카드
- ③ 뉴스·인포그래픽(텍스트+이미지 혼합)

3) 음성 기반

- ① 뉴스 오디오
- ② 음악
- ③ 팟캐스트, 오디오북

4) 영상 기반

유튜브 등 동영상 콘텐츠

3. 비정형 데이터의 특성

1. 형식 비통일
2. 매우 다양한 형태
3. 데이터 양이 방대
4. 분석 난이도 높음

비정형 데이터(Unstructured Data)는 정해진 표나 데이터베이스 형태로 저장하기 어려운 데이터이다. 텍스트, 이미지, 음성, 영상 등 다양한 형태로 존재하며 일정한 구조가 없다.

예를 들어 SNS 게시물, 사진, 음악, 유튜브 영상 등이 비정형 데이터에 해당한다.

비정형 데이터는 데이터의 양이 매우 많고 형식이 다양하여 분석이 어렵지만, 인공지능(AI), 머신러닝, 딥러닝 기술을 활용하면 의미 있는 정보를 추출할 수 있다. 최근에는 생성형 AI, 챗봇, 음성인식, 이미지 분석 등의 핵심 데이터로 활용되고 있다.

정형 / 비정형 데이터 비교

1. 정형 데이터 VS 비정형 데이터

비교 항목	정형 데이터	비정형 데이터(자연어)
구조	정해진 행과 열의 구조	일정한 구조 없음
예시	데이터베이스, 스프레드시트	텍스트, 이미지, 음성
처리 방법	데이터베이스 언어(SQL) 및 통계 분석	자연어 분석, 머신러닝
복잡성	상대적 단순	복잡함
분석 난이도	쉬움	어려움

2. 요약

1. 자연어 데이터는 구조가 없고, 형태가 다양하고, 분석이 어렵지만 중요함
2. 정형 분석 경험이 있으면 자연어 분석 이해가 빨라짐

자연어 데이터란?

자연어(Natural Language)란 사람이 실제 생활에서 사용하는 언어를 의미한다.

예를 들면 다음과 같다.

- 한국어 문장
- 영어 문장
- 뉴스 기사
- 블로그 글
- 이메일
- SNS 게시글
- 논문
- 대화 데이터

예시

오늘 날씨가 매우 좋습니다.

위 문장은 사람이 이해할 수 있는 자연어 데이터이다.

반면 컴퓨터는 문자 자체를 이해하지 못하므로 자연어를 처리 가능한 형태로 변환해야 한다.

자연어 데이터의 구성요소

1. 계층 구조

1. 글(문서)
2. 단락(Paragraph)
3. 문장(Sentence)
4. 단어(Word)
5. 구/형태소(Chunk/Morpheme)

자연어 데이터는 글 → 단락 → 문장 → 단어 → 형태소와 같은 계층 구조로 이루어져 있다. 따라서 자연어를 분석할 때는 단순히 단어만 보는 것이 아니라 문장과 문서 전체의 맥락까지 함께 고려해야 한다.

2. 다양한 데이터 포맷

1) 대화(Dialogue / Script)

- ① 등장인물 구분 필요
- ② 화자 전후 맥락 고려

2) 연설문(Speech)

3) 소설(Novel)

4) 시(Poetry)

또한 자연어 데이터는 대화문, 연설문, 소설, 시 등 다양한 형태로 존재한다. 특히 대화문은 누가 말했는지와 이전 대화 내용을 함께 분석해야 정확한 의미를 파악할 수 있다.

3. 분석 시 고려 요소

1) 화자 정보

- ① 누가 말했는지
- ② 앞 내용과의 맥락

2) 톤·어조(Prosody)

- ① "밥을 먹습니다." VS "밥을 먹습니까?"
- ② 어미 변화 → 의미 변화(서술/의문 등)

자연어 처리에서는 **화자 정보, 문맥, 톤과 어조**가 매우 중요하다. 예를 들어 "밥을 먹습니다."와 "밥을 먹습니까?"는 단어는 비슷하지만 문장 끝 어미가 달라 의미가 완전히 달라진다. 따라서 AI는 단어뿐 아니라 문맥과 표현 방식까지 함께 분석해야 정확하게 이해할 수 있다.

자연어의 고유 특징

1. 데이터·표현 다양성

1. 형식 다양: 문장, 단락, 대화, 시, 소설 등
2. 언어 다양: 여러 언어 존재
3. 표현 다양: 형태소·구문·의미 단위까지 포함

자연어 데이터는 문장, 대화, 소설, 시 등 매우 다양한 형태로 존재하며, 여러 언어와 다양한 표현 방식을 포함한다.

2. 특징적 요소

1) 중의성

- ① 표현이 하나의 의미로 고정되지 않음
 - ② 예: "그가 그린 그림"
- "그"가 누구인지 문맥 기반 해석 필요

2) 오탈자·약어

- ① 예: "ㅋㅋㅋ", " 쌤썬썬"
- ② 맞춤법·줄임말이 빈번, 비표준 표현 존재

3) 이모티콘/기호

- ① 글이 아닌 감정 등의 반응 전달 기능
- ② 예: 😊 😄 → 긍정적 피드백 의미

4) 문맥 의존성

- ① 동일 표현이 상황에 따라 의미 변화, 앞뒤 상황·맥락 필요
 - ② 예: "대박"
- 좋은 의미: "우와, 대박!"
 - 부정 의미: "대박 실수다..."

자연어의 가장 큰 특징은 **중의성**과 **문맥 의존성**이다. 같은 단어나 문장이라도 상황에 따라 의미가 달라질 수 있기 때문에 앞뒤 문맥을 함께 분석해야 한다. 또한 인터넷에서는 줄임말, 오탈자, 이모티콘 등이 자주 사용되어 분석이 더욱 복잡해진다.

5) 형태소·구문·의미 단위

- ① 문장 구성요소가 모두 의미를 갖는 것은 아님
- ② 예: "나는 오늘 학교에 갔다."
 - 의미 단위: 나 / 오늘 / 학교 / 가다
 - 조사("는", "-에")는 분석 시 의미 부여 X

자연어 처리에서는 문장을 형태소 단위로 분해하여 의미 있는 단어를 추출하고, 문맥을 고려하여 실제 의미를 파악하는 과정이 중요하다. 예를 들어 "나는 오늘 학교에 갔다."라는 문장은 "나", "오늘", "학교", "가다"와 같은 핵심 의미 단위를 중심으로 분석한다.

다양한 언어와 환경적 다양성

1. 언어별 차이

1) 문법·구조 차이: 언어마다 분석 방식 달라짐

- ① 영어: 단어 + 띄어쓰기 중심
- ② 일본어/중국어: 구조·표현 방식 상이

2) 어순 차이: 언어마다 문장 형식·어순이 달라 분석 시 구조 고려 필요

자연어는 언어마다 문법과 어순이 다르기 때문에 같은 의미라도 표현 방식이 달라진다. 따라서 자연어 처리에서는 각 언어의 특징을 고려하여 분석해야 한다.

2. 시간에 따른 변화

1) 시대·세대에 따라 표현 변화

2) 예: "대관절" → 사용 감소

3) 유행어 vs 소멸어 존재

또한 언어는 시대와 세대에 따라 계속 변화한다. 어떤 단어는 사라지고 새로운 유행어가 등장하기 때문에 최신 언어 사용 경향을 반영해야 한다.

3. 번역·뉘앙스 문제

1) 관용적 표현·문맥·감정에 따라 의미 달라짐

2) 예: "양반"

- ① 기존에는 긍정적인 의미
- ② 현대 일상에서는 부정적 뉘앙스 가능

번역 과정에서도 단어의 사전적 의미만으로는 충분하지 않다. 문맥, 감정, 문화적 배경에 따라 의미와 뉘앙스가 달라질 수 있으므로, 자연어 처리에서는 단어뿐 아니라 문장 전체의 맥락을 함께 분석하는 것이 중요하다.

자연어의 시각화

1. 특징

1) 시각화의 어려움

- ① 숫자 데이터: 단위·축으로 비교 가능
예: 키·몸무게 → 표/그래프 가능
- ② 자연어 데이터: 단위가 없어 비교 어려움

2) 대표 시각화 방식: 워드 클라우드

- ① 단어의 등장 빈도·비중을 글자 크기로 표시
- ② 자연어 표현을 시각적으로 보여주는 대표 기법

자연어 데이터는 숫자 데이터처럼 바로 그래프로 표현하기 어렵기 때문에 **워드 클라우드**와 같은 시각화 방법을 사용한다. 워드 클라우드는 많이 등장한 단어를 크게 표시하여 주요 내용을 한눈에 파악할 수 있게 해준다.

2. 데이터 분석을 위한 전처리

- 1) 원본 텍스트를 문장 → 단어 → 의미 요소로 분해
- 2) 의미 있는 요소만 남겨 분석(불용어 제거 등)

3. 음성 데이터와의 차이

- 1) 음성도 자연어 데이터가 맞음
- 2) 발음·억양 포함 → 텍스트 변환 과정 필요해 분석 방법이 다름

자연어를 분석하기 위해서는 먼저 문장을 단어와 형태소로 분해하고, 조사나 접속사와 같은 불필요한 단어(불용어)를 제거하는 **전처리 과정**이 필요하다.

또한 음성 데이터도 자연어 데이터의 한 종류이지만, 발음과 억양 정보가 포함되어 있으므로 먼저 음성을 텍스트로 변환(STT, Speech-to-Text)한 후 분석하는 경우가 많다. 따라서 텍스트 데이터와 음성 데이터는 분석 과정에 차이가 있다.

2. 텍스트 데이터 전처리

자연어 데이터 전처리 개요

1. 목적

1. 수집한 텍스트를 문맥/단어 단위로 다듬어 분석 가능하게 만드는 과정
2. 비유: 돈가스를 먹기 전에 잘 썰어놓는 과정

2. 주요 절차

1) 토큰화(Tokenization)

문장/단어 단위로 쪼개기

2) 정제(Cleaning)

불필요한 문자/기호 제거

3) 정규화(Normalization)

표현 통일

(예: "합니다" / "한다" → "하다")

4) 불용어 제거(Stopword Removal)

의미 없는 단어 제거

5) 어간/표제어 추출(Stemming/Lemmatization)

단어의 기본형으로 변환

3. 자연어 vs 정형 데이터 전처리 차이

1) 정형 데이터(엑셀 등)

전처리 절차가 비교적 간단

2) 자연어 데이터

- ① 수집부터 복잡
- ② 쪼개고/정리하고/필터링하는 과정 필요

자연어 전처리(NLP Preprocessing)는 텍스트 데이터를 분석하기 전에 깨끗하고 일관된 형태로 가공하는 과정이다.

예를 들어 "저는 밥을 먹었습니다."라는 문장이 있으면 다음과 같은 과정을 거친다.

- 토큰화 → 저는, 밥을, 먹었습니다
- 정제 → 특수문자 제거
- 정규화 → 먹었습니다 → 먹다
- 불용어 제거 → 조사 제거
- 표제어 추출 → 단어의 기본형 유지

최종적으로는 다음과 같이 변환될 수 있다.

밥

먹다

이러한 전처리를 수행하면 텍스트의 핵심 의미만 남게 되어 머신러닝, 딥러닝, LLM(대규모 언어 모델) 등이 데이터를 더 정확하게 학습하고 분석할 수 있다. 정형 데이터는 결측치 처리나 형식 변환 정도가 주된 전처리 작업이지만, 자연어 데이터는 단어 분리와 의미 추출 과정이 추가되므로 전처리 과정이 훨씬 복잡하다.

자연어 처리(NLP)란?

NLP(Natural Language Processing)는 컴퓨터가 인간의 언어를 이해하고 처리할 수 있도록 만드는 기술이다.

자연어 처리 기술은 다음과 같은 분야에서 활용된다.

- 번역 시스템
- 챗봇
- 음성 비서
- 검색 엔진
- 감성 분석
- 문서 분류
- LLM

LLM 을 학습시키기 위해서는 먼저 자연어 처리 과정을 통해 데이터를 정리해야 한다.

자연어 처리(NLP)의 정의와 범위

1. 분석 모델(Model)

1) 개념

- ① 특정 문제를 해결하도록 만든 분석용 도구/체계
- ② 예
 - 수요 모델 / 공급 모델
 - 감성 분석 모델 → 글이 칭찬인지 비난인지 판단

2) 예시 상황

- ① 뉴스가 나오면 주가가 오를지/내릴지 예측
- ② 텍스트를 분석해 긍정/부정 감정 판단 등

분석 모델(Model)은 특정 문제를 해결하기 위해 만든 분석 도구이다. 예를 들어 뉴스 기사를 분석하여 주가를 예측하거나, 댓글이 긍정적인지 부정적인지를 판단하는 감성 분석 모델이 있다.

2. 방법론(Methodology)

1) 모델을 적용하는 방식과 절차

- 2) 분석 대상 데이터가 정형 데이터(예: 가격 표)인지, 비정형 데이터(예: 글·음성·이미지)인지에 따라 달라짐

방법론(Methodology)은 이러한 모델을 실제 데이터에 적용하는 절차와 방법을 의미한다. 데이터가 정형 데이터인지 비정형 데이터인지에 따라 사용하는 분석 방법이 달라진다.

3. NLP (Natural Language Processing)

1) 정의

사람의 언어(자연어)를 컴퓨터가 이해하고 처리하게 하는 기술 분야

2) 데이터 범위

- ① 텍스트 (문서, 댓글, 뉴스 등)
- ② 음성 (말한 내용을 텍스트화)
- ③ 번역 (영↔한 등)
- ④ 감성 분석
- ⑤ 요약
- ⑥ 대화형 모델(ChatGPT 등)

NLP(자연어 처리)는 사람이 사용하는 언어를 컴퓨터가 이해하고 처리하도록 만드는 기술이다. 문서 분석, 음성 인식, 번역, 감성 분석, 문서 요약, ChatGPT 와 같은 대화형 AI 가 모두 NLP 기술을 활용한 대표적인 사례이다.

자연어 데이터 전처리의 중요성

1. 중요 이유

1. 전처리가 잘못되면 이후 분석 결과가 모두 잘못됨
2. 전처리 = 분석 품질의 기반

자연어 처리에서 **전처리(Preprocessing)**는 분석 결과의 품질을 결정하는 매우 중요한 단계이다. 전처리가 잘못되면 아무리 좋은 머신러닝이나 딥러닝 모델을 사용해도 정확한 결과를 얻기 어렵다.

2. 전처리 시 주의사항

1) 특수문자·기호 처리

- ① 단순히 모두 제거하면 안 됨
- ② 예: 😊 (긍정 표현) vs 😞 (부정 표현)
→ 이모티콘도 감정 정보를 담고 있음

2) 정보 손실 위험

잘못 제거하면 중요한 의미가 사라짐

전처리 과정에서는 특수문자, 기호, 이모티콘 등을 무조건 삭제하면 안 된다. 예를 들어 😊는 긍정 감정, 😞는 부정 감정을 나타내므로 중요한 정보가 될 수 있다.

3. 전처리 품질이 분석에 미치는 영향

- 1) 부적절한 전처리 → 정보 손실
- 2) 불필요한 데이터 방치 → 처리 속도 저하
- 3) 결국 성능·정확도 모두 악화됨

또한 불필요한 데이터를 제거하지 않으면 처리 속도가 느려지고, 반대로 중요한 정보를 제거하면 의미가 손실된다. 따라서 전처리는 **불필요한 정보는 제거하고 중요한 정보는 보존하는 과정**이라고 할 수 있으며, 이는 자연어 분석의 정확도와 성능에 직접적인 영향을 미친다.

자연어 데이터 준비 절차

초거대언어모델(LLM)은 방대한 양의 자연어 데이터를 학습하여 언어를 이해하고 생성하는 인공지능 모델이다.

LLM 을 학습시키기 위해서는 다음과 같은 자연어 처리 과정이 필요하다.

1. 데이터 수집
2. 데이터 정제
3. 문장 분리
4. 토큰화
5. 형태소 분석
6. 벡터화
7. 모델 학습
8. 모델 평가

형태소 분석은 문장을 의미 단위로 분해하여 언어 구조를 이해하는 중요한 기술이며, 특히 한국어와 같이 조사와 어미가 발달한 언어에서는 매우 중요한 역할을 수행한다.

LLM 학습을 위한 자연어 데이터 준비 과정은 일반적으로 다음과 같은 단계로 구성된다.

1 단계. 데이터 수집

다양한 출처에서 텍스트를 수집한다.

예시

- 뉴스
- 블로그
- 책
- 논문
- 웹사이트
- 위키 문서

수집된 데이터의 양이 많을수록 다양한 언어 패턴을 학습할 수 있다.

2 단계. 데이터 정제(Cleaning)

수집된 데이터에는 불필요한 정보가 포함되어 있다.

예시

```
<html>
<body>
안녕하세요.
</body>
</html>
```

정제 후

```
안녕하세요.
```

제거 대상

- HTML 태그
- 광고 문구
- 특수문자
- 중복 문장
- 깨진 문자

3 단계. 문장 분리(Sentence Segmentation)

긴 문서를 문장 단위로 나눈다.

예시

```
오늘은 날씨가 좋다. 나는 산책을 갔다.
↓
오늘은 날씨가 좋다.
나는 산책을 갔다.
```

4 단계. 토큰화(Tokenization)

문장을 작은 단위로 분리한다.

예시

```
나는 학교에 간다.
```


토큰화 결과

[나는] [학교에] [간다]

이렇게 분리된 단위를 토큰(Token)이라고 한다.

5 단계. 형태소 분석

토큰을 더욱 세부적으로 분석한다.

예시

학생들이 공부한다.

↓

학생 / 명사

들 / 접미사

이 / 조사

공부 / 명사

한다 / 동사

6 단계. 벡터화(Vectorization)

컴퓨터는 문자보다 숫자를 잘 처리한다.

예시

사과 → [0.2, 0.7, 0.5]

바나나 → [0.8, 0.1, 0.4]

이처럼 텍스트를 숫자 형태로 변환하는 과정을 벡터화라고 한다.

7 단계. 학습 데이터 구축

최종적으로 모델이 학습할 수 있는 형식으로 저장한다.

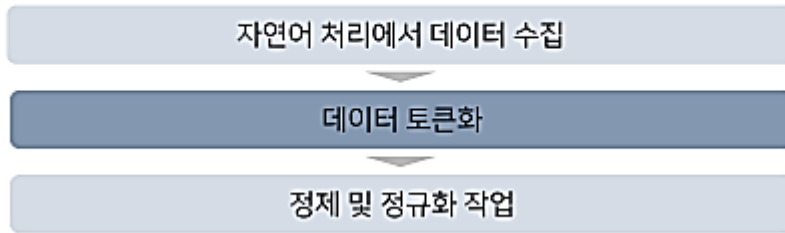
예시

입력 : 오늘 날씨는?

출력 : 매우 맑습니다.

텍스트 토큰화

❖ 텍스트 정제 과정



❖ 토큰화

- 문장이 있을 때 문장을 토큰 단위로 나누는 것
- 토큰화 수행 방법: KoNLPy, NLTK

❖ 단어 토큰화 작업 방법

■ 구두점(Punctuation)과 같은 문자의 경우

- 콤마(Comma), 점, 마침표, 특수문자 등을 제외하는 작업도 함께 수행

■ 아포스트로피(' , Apostrophe)가 들어가 있는 단어의 경우

- 아포스트로피를 빼거나 원래 단어 복원

■ 줄임말이나 띄어쓰기의 경우

- 줄임말이나 축약형은 원래의 형태로 복원 고려

텍스트 정제(Text Cleaning)는 자연어 처리에서 수집한 텍스트를 분석하기 쉬운 형태로 가공하는 과정이다.

일반적으로 다음 순서로 진행된다.

1. **데이터 수집** : 문서, 뉴스, 댓글 등의 텍스트 확보
2. **토큰화(Tokenization)** : 문장을 단어나 형태소 단위로 분리
3. **정제 및 정규화** : 불필요한 기호 제거, 표현 통일

예를 들어 다음 문장이 있다고 가정하자.

I'm happy!!! ㅋㅋㅋ

전처리 후에는 다음과 같이 변환될 수 있다.

I am happy

이 과정에서 특수문자(!), 불필요한 기호를 제거하고, 축약형(I'm)을 원래 형태(I am)로 복원한다.

토큰화에는 **KoNLPy(한국어 자연어 처리 라이브러리)**와 **NLTK(영어 자연어 처리 라이브러리)**가 많이 사용되며, 이러한 과정을 거쳐야 머신러닝, 딥러닝, LLM 이 텍스트의 의미를 보다 정확하게 분석할 수 있다.

■ 구두점 및 특수문자 유의할 점

- **점(.)**의 경우 : 문장과 문장 사이를 구분하는 경계 역할을 하므로 무조건 제거하는 것은 안 됨
- **앤드(&)**의 경우 : AT&T 의 경우 하나의 중요한 의미로 사용이 되어질 수도 있음
- **화폐 단위**의 경우 : 화폐 단위 중간 점의 유무로 큰 차이가 발생할 수 있음
- **coma(,)나 점(.)**의 경우 무조건 제거 금지

텍스트 전처리에서는 **구두점과 특수문자를 무조건 제거하면 안 된다**. 일부 기호는 중요한 의미를 포함하고 있기 때문이다.

예를 들어 마침표(.)는 문장의 경계를 나타내고, **AT&T** 의 &는 회사 이름의 일부이므로 제거하면 원래 의미가 훼손된다. 또한 숫자와 화폐 단위에서 점(.)이나 콤마(,)는 값의 크기를 결정하는 중요한 역할을 한다.

■ 예외 처리

- 고유 명사
- (예) 아놀드 트럼프의 경우 : "트럼프" 게임과 단어는 동일하나 의미가 완전히 다름
- 언더바(_) 같은 기호를 활용해 구분 가능

고유명사도 주의해야 한다. 같은 단어라도 문맥에 따라 의미가 달라질 수 있다. 예를 들어 "트럼프"는 사람 이름일 수도 있고 카드 게임을 의미할 수도 있다. 따라서 자연어 처리에서는 단순히 문자만 보는 것이 아니라 **문맥과 의미를 함께 고려하여 전처리**해야 한다.

◆ 정제 및 정규화 용어

■ 정제(Cleaning)

- 단어 집합으로부터 노이즈 데이터를 제거하는 작업

정제(Cleaning)는 텍스트 데이터에서 불필요한 문자, 특수문자, 중복 데이터 등 분석에 방해가 되는 **노이즈(Noise)** 를 제거하는 과정이다.

■ 정규화(Normalization)

- 다른 단어들을 통합해 주는 작업

❖ 정제 및 정규화 과정

■ 하나의 단어로 통일

- 대한민국, 한국, 코리아 같은 단어를 하나의 단어로 통일

■ 규칙에 따른 표기 다른 단어 통합

- USA, US 같은 단어를 하나의 단어로 통합

■ 대·소문자 통일

- 파이썬 언어의 경우 반드시 필요

정규화(Normalization)는 같은 의미를 가진 여러 표현을 하나의 형태로 통일하는 과정이다. 예를 들어 "**대한민국**", "**한국**", "**코리아**" 는 모두 같은 국가를 의미하므로 하나의 단어로 통일할 수 있다. 또한 "**USA**" 와 "**US**" 도 같은 의미이므로 하나로 통합한다.

특히 영어 데이터에서는 **대문자와 소문자를 통일**하는 작업이 중요하다. 예를 들어 Python, PYTHON, python 을 서로 다른 단어로 인식하지 않도록 python 으로 통일한다.

즉, **정제는 불필요한 데이터를 제거하는 과정이고, 정규화는 같은 의미의 데이터를 하나로 통일하는 과정**이라고 이해하면 된다. 이를 통해 데이터 품질이 향상되고 자연어 분석의 정확도가 높아진다.

■ 불필요한 단어 제거

- 의미가 없는 단어나 길이가 너무 짧은 단어 같은 불용어 제거

■ 빈도가 너무 낮은 단어 제거

■ 길이가 짧거나 의미 없는 단어 제거

자연어 데이터를 분석하기 전에 **불필요한 단어를 제거하는 정제 작업**을 수행한다. 예를 들어 조사, 접속사, 의미가 없는 단어 또는 너무 짧은 단어는 분석에 큰 도움이 되지 않으므로 제거하는 경우가 많다.

또한 데이터에 한두 번만 등장하는 단어는 분석 결과에 큰 영향을 주지 않을 수 있어 제거하기도 한다. 이를 통해 데이터의 크기를 줄이고 분석 속도를 높일 수 있다.

❖ 정규 표현식

- 'short word sub' 함수 활용 시 패턴식에 부합하는 조건을 삭제 처리
- 길이가 1 또는 2 인 단어들을 정규 표현식을 사용해 삭제
- 함수 실행 후 한 글자 또는 두 글자에 해당하는 경우 모두 비문자열로 처리

이 과정에서 **정규 표현식(Regular Expression, Regex)** 을 활용하면 특정 패턴의 단어를 쉽게 찾아 제거할 수 있다. 예를 들어 길이가 1 글자 또는 2 글자인 단어를 찾아 삭제하거나, 숫자·특수문자·이메일 주소와 같은 특정 형식의 데이터를 한 번에 처리할 수 있다.

즉, **정제 작업은 의미 있는 단어만 남겨 데이터 품질을 높이는 과정**이며, 정규 표현식은 이러한 작업을 자동화하는 강력한 도구이다.

자연어 전처리 실습

1) 영문 자연어 전처리 개요

영문 자연어 전처리는 정제(Cleaning) → 토큰화(Tokenization) → 불용어 제거(Stopword Removal) 순서로 진행된다.

(1) 정제(Cleaning)

■ 특수문자 제거

- 특수문자 :

```
!"#$%&'()*+,-./:;<=>?@[WW]^_`{|}~
```

자연어 처리에서는 이러한 특수문자들이 분석에 불필요한 경우가 많아 제거한다.

예를 들어 다음 문장이 있다고 가정하자.

```
Stay Hungry. Stay Foolish!
```

특수문자를 제거하면

```
Stay Hungry Stay Foolish
```

와 같이 변환된다.

■ 제거 대상 특수문자의 종류

```
import string
# 파이썬의 string.punctuation 은 영어 문장에서 자주 사용되는 특수문자 목록을 제공한다.
print(string.punctuation)
```

출력 결과

```
!"#$%&'()*+,-./:;<=>?@[W]^_`{|}~
```

■ 영문 자연어 전처리를 위한 코퍼스(Corpus)

```
Beneath it were the words: "Stay Hungry. Stay Foolish."
It was their farewell message as they signed off.
And I have always wished that for myself.
And now, as you graduate to begin anew, I wish that for you.
출처 : 스티브 잡스, 2005, 스탠포드대학교 졸업식
```

여기서 사용된 코퍼스(Corpus) 는 자연어 처리에서 분석 대상이 되는 텍스트 데이터 집합을 의미한다.

예제에서는 스티브 잡스의 2005 년 스탠포드대학교 졸업식 연설문 일부가 코퍼스로 사용되었다.

■ 정규식을 이용한 특수문자 제거

- 이 예제는 정규표현식(Regular Expression, Regex) 을 사용하여 텍스트를 정제하는 방법을 보여준다. 정규표현식은 자연어 전처리에서 가장 많이 사용되는 도구 중 하나이다.
- **re.sub**('^[^WwW.Ws]', '', text)를 이용하여 .을 제외한 특수문자 제거
 - Ww : 영문자, 숫자, 언더바(_)
 - W. : 마침표(.)
 - Ws : 공백 문자
 - ^ : 제외(부정) 의미

```
import re

cleaned_text = re.sub('^[^WwW.Ws]', '', text)
print(cleaned_text)
```

즉, 영문자·숫자·공백·마침표를 제외한 모든 특수문자를 제거한다.

출력 결과

```
Beneath it were the words Stay Hungry.
Stay Foolish. It was their farewell message as they signed off.
And I have always wished that for myself.
And now as you graduate to begin anew I wish that for you..
```

■ 정규식을 이용한 'Wn' 문자 제거

줄바꿈 문자 제거

- **re.sub**('Wn', ' ', cleaned_text)를 이용하여 Wn 문자 제거
 - Wn 은 줄바꿈(New Line) 문자이다.
 - 줄바꿈을 공백으로 변경하여 여러 줄의 문장을 한 줄로 연결한다.

```
cleaned_text = re.sub('Wn', ' ', cleaned_text)
print(cleaned_text)
```

출력 결과

Beneath it were the words Stay Hungry. Stay Foolish. It was their farewell message as they signed off. And I have always wished that for myself. And now as you graduate to begin anew I wish that for you.

■ 대소문자 통일

- KOREA, Korea, korea → korea

텍스트에서 특수문자나 불필요한 기호를 제거하고, 대문자와 소문자를 하나로 통일하는 과정이다.

소문자화(Lower Capitalization)는 영어 자연어 처리에서 자주 수행하는 정제(Cleaning) 작업 중 하나이다.

영어에서는 같은 단어라도 대문자와 소문자가 다르면 서로 다른 단어로 인식될 수 있다. 따라서 모든 단어를 소문자로 통일하여 같은 단어를 하나로 처리한다.

■ lower() 함수와 List Comprehension 을 이용한 소문자화

```
print('소문자화 수행전 : ', tokens)

tokens = [token.lower() for token in tokens]

print('소문자화 수행후 : ', tokens)
```

코드에서 사용한 lower() 함수는 문자열을 소문자로 변환하며,

```
tokens = [token.lower() for token in tokens]
```

는 리스트 내의 모든 단어에 대해 lower()를 적용하는 리스트 컴프리헨션(List Comprehension) 문법이다.

출력 예시

```
소문자화 수행전 :
['Beneath', 'words', 'Stay', 'Hungry', 'Stay']

소문자화 수행후 :
['beneath', 'words', 'stay', 'hungry', 'stay']
```


소문자화를 수행하는 이유

1. 같은 단어를 하나로 통합하여 단어 수를 줄일 수 있다.
2. 단어 빈도 계산의 정확도를 높일 수 있다.
3. 머신러닝 및 딥러닝 모델의 학습 성능을 향상시킬 수 있다.
4. 불필요하게 중복되는 단어를 제거할 수 있다.

예를 들어 다음과 같은 데이터가 있다.

Python
python
PYTHON
Python

소문자화 전에는 3 개의 다른 단어로 인식될 수 있지만,

python
python
python
python

소문자화 후에는 하나의 단어로 통합되어 단어 빈도 분석, 워드클라우드 생성, 감성 분석 등의 결과가 더욱 정확해진다.

(2) 토큰화(Tokenization)

■ 코퍼스(Corpus)에서 분리자(Separator)를 포함하지 않는 연속적인 문자열 단위로 분리

■ 토큰화 단위에 따른 분류

- 문장(Sentence) 단위 토큰화
- 단어(Word) 단위 토큰화

■ 토큰화 단위에 따른 분류

- 어절 단위 : 띄어쓰기 단위, 영문 적합
- 형태소 단위 : 한글 적합
- Subword : 형태소와 유사, 의미 대신 통계적 방법 적용

문장을 분석하기 위해 작은 단위로 나누는 과정이다.

예시:

I love natural language processing.

단어 토큰화 결과:

```
I  
love  
natural  
language  
processing
```

■ nltk 를 이용한 영문 토큰화를 위해 punkt 모듈 다운로드 및 활용

이 예제는 NLTK(Natural Language Toolkit) 라이브러리를 사용하여 영어 문장을 문장 단위와 단어 단위로 나누는 토큰화(Tokenization) 과정을 설명한다.

• nltk punkt 모듈 다운로드

```
import nltk  
  
nltk.download('punkt')
```

punkt 는 문장의 시작과 끝을 구분하는 규칙이 저장된 NLTK 데이터이다. 영어 문장을 정확하게 분리하기 위해 필요하다.

출력 결과

```
[nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data] Unzipping tokenizers/punkt.zip.  
True
```

■ nltk.sent_tokenize 를 이용한 문장 단위 토큰화 수행

```
sent_tokens = nltk.sent_tokenize(cleaned_text)  
# 하나의 긴 문서를 여러 개의 문장으로 분리한다  
print(sent_tokens)
```

예시:

```
I love AI. I study NLP.  
↓  
['I love AI.', 'I study NLP.']
```

문서 요약, 문장 분석, 문장 분류 등에 활용된다.

출력 예시

```
[  
  'Beneath it were the words Stay Hungry.',  
  'Stay Foolish.',  
  'It was their farewell message as they signed off.',  
  ...  
]
```

■ nltk.word_tokenize 를 이용한 단어 단위 토큰화 수행

```
tokens = nltk.word_tokenize(cleaned_text)  
  
tokens
```

문장을 단어 단위로 분리한다.

예시:

```
I love AI.  
↓  
['I', 'love', 'AI', '.']
```

단어 빈도 분석, 워드클라우드 생성, 머신러닝 입력 데이터 생성 등에 활용된다.

출력 예시

```
[  
  'Beneath',  
  'it',  
  'were',  
  'the',  
  'words',  
  'Stay',  
  'Hungry',  
  ',',  
  'Stay',  
  'Foolish',  
  ',',  
  ...  
]
```

즉, 토큰화(Tokenization) 는 자연어 처리의 가장 기본 단계로, 문서를 문장이나 단어 단위로 분리하여 컴퓨터가 텍스트를 분석할 수 있도록 만드는 과정이다. 이후 불용어 제거, 형태소 분석, 단어 벡터화, 머신러닝·딥러닝 모델 학습 단계로 이어진다.

(3) 불용어 제거(Stopword Elimination)

■ 전치사, 관사 등 문장이나 문서의 특징을 표현하는데 불필요한 단어를 제거하는 단계
예)

you, my, the, a, of, at

자주 등장하지만 의미 전달에 큰 도움이 되지 않는 단어를 제거하는 과정이다.

예시:

```
I am studying the NLP at school.
```

불용어 제거 후:

```
studying NLP school
```

■ nltk 패키지의 stopwords 다운로드

```
nltk.download('stopwords')
```

출력 결과

```
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data] Unzipping corpora/stopwords.zip.  
True
```

■ nltk 패키지의 stopwords 중 영문 불용어를 로드하고 출력

```
from nltk.corpus import stopwords  
  
stop = stopwords.words('english')  
print(stop)
```

출력 예시

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours',
```

```
'ourselves', 'you', 'your', 'yours', ...]
```

■ nltk 패키지의 stopword 와 Python List Comprehension 을 이용한 불용어 제거

```
tokens = [token for token in tokens if token not in stop]

print(tokens)
```

출력 예시

```
['Beneath', 'words', 'Stay', 'Hungry', '.',
 'Stay', 'Foolish', '.', ...]
```

불용어는 문장에 자주 등장하지만 의미 전달에 큰 영향을 주지 않는 단어이다.

예시:

```
I am studying NLP at school.
```

불용어 제거 후:

```
studying NLP school
```

대표적인 영문 불용어는 다음과 같다.

```
i, me, my, we, you, the, a, an, of, at, in
```

NLTK 의 stopwords 사전을 이용하여 쉽게 제거할 수 있다.

■ lower() 함수와 List Comprehension 을 이용한 소문자화

```
print('소문자화 수행전 : ', tokens)

tokens_lower = [token.lower() for token in tokens]

print('소문자화 수행후 : ', tokens_lower)
```

출력 예시

```
소문자화 수행전 :
['Beneath', 'words', 'Stay', 'Hungry', 'Stay', 'Foolish']
```

소문자화 수행후 :

```
['beneath', 'words', 'stay', 'hungry', 'stay', 'foolish']
```

■ 불용어 제거 후 단어의 길이가 2 글자 이하인 단어 제거

불용어 제거 후에도 의미가 약한 짧은 단어가 남을 수 있다.

```
tokens = [token for token in tokens if len(token) >= 3]
```

```
print(tokens)
```

출력 예시

```
['Beneath', 'words', 'Stay',  
'Hungry', 'Stay', 'Foolish',  
'farewell', ...]
```

위 코드는 길이가 3 글자 이상인 단어만 남긴다.

예시:

AI is fun and useful

↓

fun

useful

(조건에 따라 AI 와 같은 중요한 단어는 예외 처리 가능)

전처리 단계	목적
Stopwords 다운로드	불용어 사전 준비
불용어 제거	의미 없는 단어 제거
소문자화	같은 단어를 하나로 통일
짧은 단어 제거	분석에 불필요한 단어 제거

즉, 불용어 제거 → 소문자화 → 짧은 단어 제거 과정을 거치면 텍스트의 핵심 단어만 남게 되어 워드클라우드, 감성분석, 문서분류, 머신러닝, 딥러닝, LLM 학습에 더 적합한 데이터가 된다.

한글 자연어 전처리 실습

1) 한글 자연어 전처리 개요

한글 자연어 처리에서도 가장 중요한 전처리 과정은 **토큰화(Tokenization)** 와 **불용어 제거(Stopword Elimination)** 이다.

(1) 토큰화 (Tokenization)

■ 코퍼스(Corpus)에서 분리자(Separator)를 포함하지 않는 연속적인 문자열 단위로 분리

■ 토큰화 단위에 따른 분류

- 문장(Sentence) 단위 토큰화
- 단어(Word) 단위 토큰화

■ 단어 단위 토큰화 방법에 따른 분류

- 어절 단위 : 띄어쓰기 단위, 영문에 적합
- 형태소 단위 : 한글에 적합
- Subword : 형태소와 유사, 의미 대신 통계적 방법 적용

토큰화는 문장을 분석하기 쉬운 작은 단위로 나누는 과정이다.

예를 들어,

나는 오늘 학교에 갔다.

를 띄어쓰기 기준으로 나누면

나는 / 오늘 / 학교에 / 갔다

가 된다.

하지만 한국어는 조사와 어미가 함께 붙어 있기 때문에 단순 띄어쓰기만으로는 의미를 정확히 분석하기 어렵다.

따라서 한글에서는 **형태소 단위 토큰화**를 주로 사용한다.

예시:

나는 오늘 학교에 갔다.

↓

나 / 는 / 오늘 / 학교 / 에 / 가다

이처럼 단어를 의미 단위인 형태소로 분리하여 분석한다.

(2) 불용어 제거 (Stopword Elimination)

■ 문장이나 문서의 특징을 표현하는데 불필요한 단어를 제거하는 단계

- 불용어 사전을 활용하여 제거
- 불용어 사전 관리

불용어는 자주 등장하지만 의미 전달에 큰 영향을 주지 않는 단어이다.

예를 들어,

나는 오늘 학교에 갔다.

에서

는, 에

와 같은 조사들은 분석 목적에 따라 제거할 수 있다.

불용어 제거 후:

나 / 오늘 / 학교 / 가다

한글과 영어 토큰화의 차이

구분	영어	한글
기본 단위	띄어쓰기(어절)	형태소
예시	I love AI	나는 AI 를 좋아한다
분석 방법	Word Tokenization	형태소 분석
주요 도구	NLTK, spaCy	KoNLPy, Mecab, Okt

정리

- **토큰화** : 문장을 분석 가능한 단위로 분리하는 과정
 - **어절 단위 토큰화** : 띄어쓰기 기준 분리
 - **형태소 단위 토큰화** : 의미 단위로 분리하며 한국어에 적합
- **불용어 제거** : 조사, 접속사 등 불필요한 단어 제거
- 한국어는 조사와 어미가 발달했기 때문에 영어보다 전처리 과정이 더 중요하며, 형태소 분석이 자연어 처리의 핵심 단계이다.

2) 한글 자연어 전처리 적용

(1) 정제(Cleaning)

특수문자 제거

■ 제거 대상 특수문자의 종류 - 영문과 동일

```
import string

print(string.punctuation)
```

출력 결과

```
!"#$%&'()*+,-./:;<=>?@[W]^_`{|}~
```

한글 문장에도 따옴표("), 쉼표(,), 마침표(.) 등의 특수문자가 포함될 수 있다.

■ 한글 자연어 전처리를 위한 코퍼스(Corpus)

그 밑에는 "계속 배고픔을 느끼세요. 계속 바보로 남으세요" 라는 문구가 새겨져 있었습니다. 그들이 전한 마지막 인사말이었습니다. 계속 배고픔을 느끼세요. 계속 바보로 남으세요. 그리고 저는 항상 제 자신이 그렇길 바랬습니다. 이제는 졸업을 하고 새로운 출발을 하는 여러분에게 바라는 바 입니다.

출처 : 스티브 잡스, 2005, 스탠포드대학교 졸업식

■ 정규식을 이용한 특수문자 제거

- `re.sub('[^WwWs]', '', text)`를 이용하여 특수문자 제거

```
import string
import re

cleaned_text = re.sub('[^WwWs]', '', text)
print(cleaned_text)
```

출력 예시

```
그 밑에는 계속 배고픔을 느끼세요
계속 바보로 남으세요 라는 문구가 새겨져 있었습니다
그들이 전한 마지막 인사말이었습니다
계속 배고픔을 느끼세요 계속 바보로 남으세요
그리고 저는 항상 제 자신이 그렇길 바랬습니다
이제는 졸업을 하고 새로운 출발을 하는 여러분에게 바라는 바 입니다
```

■ 정규식을 이용한 '\n' 문자 제거

- `re.sub('\n', ' ', cleaned_text)`를 이용하여 줄바꿈 문자 제거

```
cleaned_text = re.sub('\n', ' ', cleaned_text)
print(cleaned_text)
```

출력 예시

```
그 밑에는 계속 배고픔을 느끼세요 계속 바보로 남으세요 라는 문구가 새겨져 있었습니다 ...
```

문서에는 줄바꿈(\n)이 많이 포함되어 있다.

줄바꿈을 공백으로 바꾸어 하나의 문장처럼 처리할 수 있도록 만든다.

띄어쓰기가 안되어 있는 경우 띄어쓰기 적용

■ 띄어쓰기가 적용되지 않은 상태로 코퍼스 변환

- 정규식을 이용하여 띄어쓰기가 적용되지 않은 상태로 코퍼스 변환한 경우

```
non_space = re.sub('\s', '', text)
print(non_space)
```

출력 예시

```
그밑에는"계속배고픔을느끼세요.계속바보로남으세요"라는문구가새겨져있었습니다
```

한국어에서는 띄어쓰기가 잘못되어 있는 경우가 많다.

예시:

```
오늘날씨가정말좋습니다
↓
오늘 날씨가 정말 좋습니다
```

이처럼 띄어쓰기를 자동으로 복원하면 형태소 분석과 감성 분석의 정확도를 높일 수 있다.

■ 띄어쓰기 적용을 위한 PyKoSpacing 설치

- PyPI.org 에 등록되지 않은 상태이므로 git 을 이용한 설치

```
!pip install git+https://github.com/haven-jeon/PyKoSpacing.git
```

PyKoSpacing 은 딥러닝 기반 한국어 띄어쓰기 복원 도구이다.

■ PyKoSpacing 을 이용한 띄어쓰기 적용

- spacing() 함수를 이용하여 띄어쓰기 적용

```
from pykospacing import Spacing

spacing = Spacing()
new_sent = spacing(non_space)
print(new_sent)
```

출력 예시

그 밑에는 "계속 배고픔을 느껴세요. 계속 바보로 남으세요"라는 문구가 새겨져 있었습니다.

한국어는 영어와 달리 조사, 어미, 띄어쓰기 오류가 많기 때문에 정제 과정이 매우 중요하다. 특히 띄어쓰기 복원은 형태소 분석과 감성 분석의 성능을 크게 향상시키는 핵심 전처리 작업 중 하나이다.

(2) 토큰화(Tokenization)

한글 문장 단위 토큰화

- 한글 문장 단위 토큰화를 위해 kss 모듈 설치 및 활용
 - 한글 문장 단위 토큰화 적용을 위해 kss 모듈 설치

```
!pip install kss
```

- kss 모듈을 활용한 한글 문장 단위 토큰화 적용
 - kss.split_sentences()를 이용하여 한글 문장 단위 토큰화 적용

```
import kss

sent_tokens = kss.split_sentences(cleaned_text)
print(sent_tokens)
```

kss 는 한국어 문장을 문장 단위로 나누는 라이브러리이다.

예를 들어 긴 글이 있을 때 마침표, 물음표, 문장 구조를 기준으로 문장별 리스트로 분리한다.

한글 단어 단위 토큰화

- 한글 단어단위 토큰화 적용 - 어절 토큰화

```
def tokenizer(words):  
    tokens = words.split()  
    return tokens  
  
tokens = tokenizer(cleaned_text)  
print(tokens)
```

split() 함수는 공백을 기준으로 문장을 나눈다.

한국어에서는 이를 **어절 토큰화**라고 볼 수 있다.

예를 들어 "그 밑에는 계속 배고픔을 느끼세요"는 "그", "밑에는", "계속", "배고픔을"처럼 공백 기준으로 분리된다.

(3) 불용어 제거(Stopword Elimination)

한글 불용어 사전 작성 및 업로드

- 한글 불용어 사전 작성 및 stopwords_dict.csv 로 저장

예시 CSV 내용:

	A
1	stopword
2	그
3	라는
4	바
5	입니다
6	하고

- 한글 불용어 사전 업로드

```
from google.colab import files
import os

data_dir = 'data'

if not os.path.exists(data_dir):
    os.mkdir(data_dir)

os.chdir(data_dir)
files.upload()
os.chdir('..')
```

불용어는 분석에 큰 의미가 없는 단어이다.

예를 들어 "그", "라는", "입니다", "하고" 같은 단어는 문장에 자주 나오지만 핵심 의미를 파악하는 데는 큰 도움이 되지 않을 수 있다.

따라서 CSV 파일로 불용어 목록을 만들어 업로드한다.

- pandas 패키지를 이용하여 업로드된 불용어 사전 출력

```
import pandas as pd

stop = pd.read_csv('./data/stopword_dict.csv')
print(stop[:10])
```

출력 예시:

```
stopword
0      그
1     라는
2      바
3     입니다
4     하고
```

- pandas 패키지를 이용하여 업로드된 불용어 사전 리스트로 변환

```
print(list(stop.stopword))
```

출력 예시:

```
['그', '라는', '바', '입니다', '하고']
```

pd.read_csv()는 CSV 파일을 읽어 데이터프레임으로 변환한다.

그 후 list(stop.stopword)를 사용하면 불용어 컬럼의 값을 파이썬 리스트로 바꿀 수 있다.

이 리스트는 이후 토큰 목록에서 불용어를 제거할 때 사용된다.

- 파이썬 List Comprehension 을 이용하여 불용어 제거

```
tokens = [  
    token for token in tokens  
    if token not in list(stop.stopword)  
]  
  
print(tokens)
```

이 코드는 tokens 리스트 안의 단어를 하나씩 확인하면서 불용어 목록에 없는 단어만 다시 저장한다.

예를 들어 불용어 목록에 "그"가 있다면 "그"는 제거되고, "배고픔을", "느끼세요"처럼 의미가 있는 단어만 남는다.

즉, 전체 흐름은 다음과 같다.

```
원문 텍스트  
→ 문장 단위 토큰화  
→ 단어 단위 토큰화  
→ 불용어 사전 불러오기  
→ 불용어 제거  
→ 분석에 필요한 단어만 남김
```

이 과정은 자연어 처리에서 문장을 더 깔끔하게 정리하고, 중요한 단어 중심으로 분석하기 위한 전처리 단계이다.

3. 형태소 분석

형태소 분석이란?

형태소(Morpheme)는 의미를 가지는 가장 작은 언어 단위이다.

예시

학생들이

분석 결과

학생	→ 명사
들	→ 복수 접미사
이	→ 조사

즉, 하나의 단어 안에도 여러 개의 형태소가 존재할 수 있다.

형태소 분석이 필요한 이유

한국어는 조사와 어미가 매우 다양하다.

예시

먹다
먹는다
먹었다
먹겠습니다

모두 같은 동사 "먹다"에서 파생된 형태이다.

형태소 분석을 수행하면 이러한 단어들을 동일한 의미 계열로 처리할 수 있다.

활용 분야

- 검색 엔진
- 챗봇
- 감성 분석
- 문서 분류
- 정보 추출
- LLM 데이터 전처리

한국어 형태소 분석 예시

문장

오늘은 날씨가 매우 좋습니다.

형태소 분석 결과

형태소	품사
오늘	명사
은	조사
날씨	명사
가	조사
매우	부사
좋	형용사 어간
습니다	어미

이 과정을 통해 컴퓨터는 문장의 문법 구조를 이해할 수 있다.

품사(Part of Speech)란?

품사는 단어의 역할을 의미한다.

주요 품사

품사	설명
명사	사람, 사물 이름
동사	행동
형용사	상태
부사	정도 설명
조사	문법 관계
어미	문장 종결

예시

철수가 빨리 달린다.

분석

철수 → 명사
가 → 조사
빨리 → 부사
달린다 → 동사

LLM 에서 형태소 분석의 역할

초기 자연어 처리 시스템은 형태소 분석에 크게 의존하였다.

최근의 LLM 은 형태소 대신 서브워드(Subword) 기반 토큰라이저를 많이 사용한다.

대표적인 방식

BPE(Byte Pair Encoding)

자주 등장하는 문자 조합을 하나의 토큰으로 결합

예시

play
playing
played
↓
play + ing
play + ed

WordPiece

자주 사용되는 단어 조각을 학습

예시

unhappy
↓
un + happy

SentencePiece

공백을 포함하여 토큰을 생성하는 방식
한국어 처리에서도 많이 사용된다.

자연어 데이터 품질 향상 방법

좋은 LLM 을 만들기 위해서는 좋은 데이터가 필요하다.

품질 향상 방법

- **중복 데이터 제거**
동일 문장이 반복되지 않도록 한다.
- **오타자 수정**
안녕하세여
↓
안녕하세요
- **광고 제거**
지금 구매하면 90% 할인!
같은 불필요한 광고 제거
- **데이터 균형 유지**
특정 주제만 과도하게 많지 않도록 조정

1. 영문·한글 형태소 분석

1) 영문 형태소 분석

(1) 형태소 분석을 위한 전처리 수행



(2) 자연어 처리를 위한 NLTK

nltk 를 이용한 영문 형태소 분석

- pos_tag() 함수를 이용하여 품사 태깅 수행

```
from nltk.tag import pos_tag

tagged_list = pos_tag(set(tokens))
print(tagged_list)
```

영문 형태소 분석의 기본 과정이다. 형태소 분석 전에 특수문자 제거, 토큰화, 불용어 제거, 소문자화 등의 전처리를 수행하며, NLTK의 pos_tag() 함수로 단어별 품사를 부여한다.

2) 한글 형태소 분석

(1) 형태소 분석을 위한 전처리 수행



(2) KoNLPy 및 관련 패키지 설치

Java 설치

- JPyype 설치

```
pip install JPyype1
```

- KoNLPy 설치

```
pip install konlpy
```

- Linux 용 Mecab 설치

```
!pip install JPyype1
!pip install konlpy
!sudo apt-get install curl git
!bash <(<curl -s https://raw.githubusercontent.com/konlpy/konlpy/master/s
```

(3) KoNLPy 를 이용한 한글 형태소 분석

형태소 분석기 객체 생성

사용 가능한 형태소 분석기

- Okt
- Kkma
- Komoran
- Hannanum
- Mecab

```
from konlpy.tag import Okt, Kkma, Komoran, Hannanum, Mecab

# tagger = Kkma()
# tagger = Komoran()
# tagger = Hannanum()
# tagger = Okt()
tagger = Mecab()
```

■ 형태소 분석 클래스별 품사 태깅 성능 비교

Hannanum	Kkma	Komoran	Mecab	Twitter
아버지가방에 들어가 / N	아버지 / NNG	아버지가방에 들어가신다 / NNP	아버지 / NNG	아버지 / Noun
이 / J	가방 / NNG		가 / JKS	가방 / Noun
시~다 / E	에 / JKM		방 / NNG	에 / Josa
	들어가 / VV		에 / JKB	들어가신 / Verb
	시 / EPH		들어가 / VV	다 / Eomi
	s다 / EFN		신다 / EP+EC	

■ 형태소 분석 클래스별 수행시간 성능 비교

Hannanum	Kkma	Komoran
Kkma	5.6988 secs	35.7163 secs
Komoran	5.4866 secs	25.6008 secs
Hannanum	0.6591 secs	8.8251 secs
Okt(previous Twitter)	1.4870 secs	2.4714 secs
Mecab	0.0007 secs	0.2838 secs

※ 출처 : KoNLPy 형태소 분석 및 품사 태깅,
<https://konlpy-ko.readthedocs.io/ko/v0.4.3/morph/>

■ morphs 함수를 이용한 형태소 분리

```
tag_list = tagger.morphs(cleaned_text)
print(tag_list)
```

['그', '밑', '에', '는', '계속', '배고픔', '을', '느끼', '세요',

■ pos 함수를 이용한 형태소 분리 및 품사 태깅

```
pos_list = tagger.pos(cleaned_text)
print(pos_list)
```

[('그', 'MM'), ('밑', 'NNG'), ('에', 'JKB'), ('는', 'JX'),

- **nouns** 함수를 이용한 명사 추출 `tagger.nouns(text)`

```
noun_list = tagger.nouns(cleaned_text)
print(noun_list)

['밀', '배고픔', '바보', '문구', '그', '마지막', '인사말', '배고픔']
```

2. 형태소 분석 사전 활용

1) 신조어 형태소 분석

“최근 정부 주도 디지털뉴딜 사업인 **확장현실(XR)** 메타버스 프로젝트는 **증강현실(AR)** 플랫폼 분야의 주도 기업이 추진할 것으로 예상된다.”



확장현실(XR)

메타버스

증강현실(AR)

- 메타버스 등과 같은 신조어 형태소 분석은?

- ① 다양한 형태소 분석 방법 활용 - SoyNLP 및 ETRI 형태소 분석 Open API 등 활용
- ② 형태소 분석기 사전 활용 - 형태소 분석기 사전에 신조어나 도메인 특화 용어 등록
- ③ 모델 학습을 통해 신조어나 도메인 특화 용어 학습

2) KoalaNLP 패키지 활용

(1) KoalaNLP 개요

- 한국어 형태소 및 구문 분석기 모음
 - KoNLPy, ETRI 형태소 분석, OpenAPI, Khaiii, Kiwi 등

(2) KoalaNLP 설치

- pip 를 이용한 koalanlp 설치

```
pip install koalanlp
```

(3) KoalaNLP 지원 형태소 분석기

분석기별 접근키	설명	분석 범위
API.KMR	코모란 Wrapper	형태소
API.EUNJEON	은전한닢 Wrapper	형태소
API.ARIRANG	아리랑 Wrapper	형태소
API.RHINO	RHINO Wrapper	형태소
API.DAON	Daon Wrapper	형태소 2-3
API.KHAIIL	Khaili Wrapper	형태소 2-4
API.UTAGGER	울산대 UTagger Wrapper	형태소, 문장분리
API.OKT	Open Korean Text Wrapper	형태소, 의존구문
API.KKMA	꼬꼬마 Wrapper	형태소, 의존구문
API.HNN	한나눔 Wrapper	문장분리, 형태소, 구문분석, 의존구문
API.ETRI	ETRI Open API Wrapper	형태소, 의존구문, 구문분석, 개체명, 의미역
API.KSS	KSS Wrapper	문장분리
API.KIWI	Kiwi Wrapper	형태소

※출처 : <https://pypi.org/project/koalanlp/>

(4) KoalaNLP 지원 형태소 분석기 사용

- import
 - 파이썬에서 사용하기 위한 모듈, 클래스, 함수 import
- initialize
 - 다운로드 및 자원 설정 초기화
- 형태소 분석
 - KoalaNLP 지원 형태소 분석기 사용
- finalize
 - 사용자원 반납 등 사용종료 처리

(5) 형태소 분석을 위한 코퍼스(Corpus)

예시 문장

```
text = """최근 정부 주도 디지털뉴딜 사업인 확장현실(XR) 메타버스 프로젝트는 증강현실(AR) 플랫폼 분야의 주도 기업이 추진할 것으로 예상된다."""
```

■ 파이썬에서 사용하기 위한 모듈, 클래스, 함수 import

```
from koalanlp.Util import initialize, finalize
from koalanlp.proc import *
from koalanlp import API
```

■ 다운로드 및 자원 설정 초기화

```
initialize(java_options="-Xmx4g -Dfile.encoding=utf-8",
```

형태소 분석에 사용할 문장(코퍼스)을 정의하고 KoalaNLP 초기 설정을 수행한다.

■ KoalaNLP 지원 형태소 분석기 사용 - eunjeon (은전)

```
tagger = Tagger(API.EUNJEON)

tagged = tagger.tag(text)

print(tagged[0].singleLineString())
```

```
최근/NNG 정부/NNG 주도/NNG 디지털/NNG 뉴딜/NNG 사업
/NNG+이/VCP+L/ETM
확장/NNG 현실/NNG (/SS XR/SL)/SS 메타/NNG 버스/NNG 프
로젝트/NNG+는/JX
증강/NNG+현실/NNG
```

은전한닢 분석기는 신조어를 잘 모르기 때문에 "메타버스"를 "메타"+"버스"로 분리한다.

▪ KoalaNLP 지원 형태소 분석기 사용 - ETRI

- ETRI Key 발급 :

공공 인공지능 오픈 API·DATA 서비스 포털 접속 후 이메일 인증 및 키 발급 신청

- https://aiopen.etri.re.kr/key_main.php 절차

1. 이메일 입력
2. 인증메일 수신
3. 인증키 입력
4. API 키 발급

```
tagger = Tagger(API.ETRI, etri_key= '발급받은 키 입력')  
  
tagged = tagger.tag(text)  
  
print(tagged[0].singleLineString())
```

최근/NNG 정부/NNG 주도/NNG 디지털/NNG+뉴딜/NNG 사업
/NNG+이/VCP+L /ETM
확장/NNG+현실/NNG+(/SS+XR/SL+)/SS 메타버스/NNG 프로
젝트/NNG+는/JX
증강/NNG+현실/NNG+(/SS+AR/SL+)/SS

▪ eunjeon 형태소 분석기 사용자 사전 등록

```
from koalanlp.types import POS  
from koalanlp.proc import Dictionary  
  
dictionary = Dictionary(API.EUNJEON)  
  
dictionary.addUserDictionary(("메타버스", POS.NNG))  
dictionary.addUserDictionary(("확장현실", POS.NNG), ("메타버스", POS.NNG))  
  
print(dictionary.getItems())  
  
[('메타버스', NNG), ('확장현실', NNG)]
```

ETRI 는 신조어를 하나의 단어로 인식한다. 또한 사용자 사전에 직접 단어를 등록할 수 있다.

▪ eunjeon 형태소 분석 수행

```
tagger = Tagger(API.EUNJEON)  
  
tagged = tagger.tag(text)  
  
print(tagged[0].singleLineString())
```

사용자 사전에 신조어를 등록하면 은전한닢 분석기도 신조어를 올바르게 분석할 수 있다.

▪ 사용자 자원 반납 등 사용종료 처리를 위한 finalize

```
finalize()
```

```
True
```

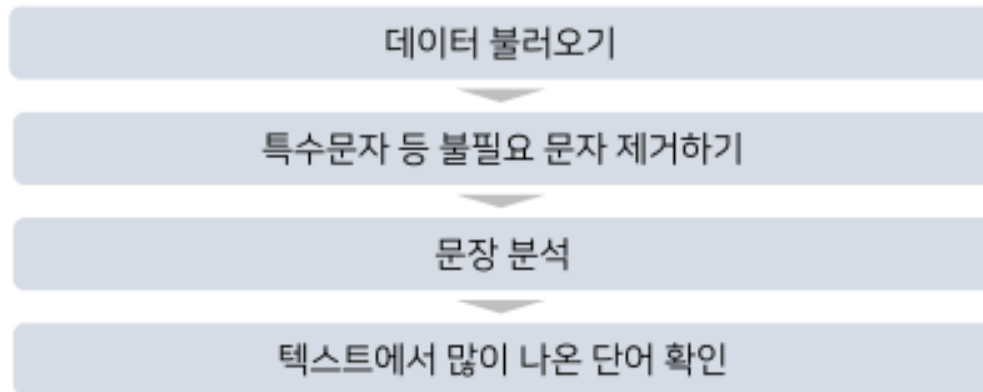
KoalaNLP 사용이 끝난 후 메모리와 자원을 정리하기 위해 finalize()를 호출한다.

요약 정리

1. 영문 형태소 분석
 - NLTK의 pos_tag() 사용
 - 품사 태깅 수행
2. 한글 형태소 분석
 - KoNLPy 설치
 - Okt, Kkma, Komoran, Hannanum, Mecab 활용
3. 형태소 분석 함수
 - morphs()
 - pos()
 - nouns()
4. 신조어 처리
 - 메타버스, 확장현실(XR), 증강현실(AR)
 - 사용자 사전 등록
5. KoalaNLP 활용
 - 여러 형태소 분석기 통합 사용
 - ETRI Open API 연동
 - 사용자 사전 구축
6. 형태소 분석 성능 비교
 - 정확도와 처리 속도 비교
 - Mecab이 가장 빠른 성능을 제공
7. 사용자 사전 활용
 - 신조어 및 전문용어를 등록하여 분석 정확도 향상
8. 자원 관리
 - initialize()
 - finalize()를 이용한 분석 환경 초기화 및 종료 처리.

4. 워드 클라우드 시각화

❖ 단어 빈도 시각화 과정



단어 빈도 시각화 과정을 4 단계로 정리한 흐름도입니다.

1. 데이터 불러오기

첫 번째 단계는 분석할 텍스트 데이터를 가져오는 과정입니다.

예를 들어 소설, 뉴스 기사, 리뷰 데이터, 댓글 데이터, 설문 응답, 고객 상담 내용 등을 파일에서 읽어옵니다.

파이썬에서는 보통 다음과 같이 파일을 불러옵니다.

```
f = open("text.txt", "r", encoding="utf-8")
lines = f.readlines()
f.close()
```

여기서 open()은 파일을 여는 함수이고, "r"은 읽기 모드를 의미합니다.

encoding="utf-8"은 한글이 깨지지 않도록 문자 인코딩을 지정하는 부분입니다.

readlines()는 텍스트 파일의 여러 줄을 리스트 형태로 읽어오는 함수입니다.

즉, 이 단계에서는 컴퓨터가 분석할 수 있도록 텍스트 원본을 메모리로 가져옵니다.

2. 특수문자 등 불필요 문자 제거하기

두 번째 단계는 텍스트에서 분석에 필요 없는 문자를 제거하는 과정입니다.

실제 텍스트 데이터에는 다음과 같은 불필요한 요소가 많이 포함되어 있습니다.

ㅋㅋㅋ, ㅎㅎ, !!!, @, #, %, 숫자, 영어, HTML 태그, 이모지

예를 들어 다음 문장이 있다고 하겠습니다.

안녕하세요!!! 오늘 날씨가 정말 좋네요^^ 2026 년

불필요한 문자를 제거하면 다음과 같이 정리할 수 있습니다.

안녕하세요 오늘 날씨가 정말 좋네요 년

파이썬에서는 정규표현식을 사용하여 정리할 수 있습니다.

```
import re

text = re.sub(r"[^ㄱ-ㅎㅏ-ㅣ가-힣ws]", " ", text)
```

이 코드는 한글과 공백을 제외한 문자들을 공백으로 바꾸는 코드입니다.

이 과정을 수행하는 이유는 불필요한 문자가 많으면 단어 빈도 분석 결과가 부정확해질 수 있기 때문입니다.

3. 문장 분석

세 번째 단계는 정리된 문장을 단어 단위로 분석하는 과정입니다.

한국어는 영어처럼 단순히 공백 기준으로 단어를 나누기 어렵습니다.

예를 들어 다음 문장을 보겠습니다.

학생들이 공부한다

공백 기준으로 보면 다음과 같습니다.

학생들이
공부한다

하지만 형태소 분석을 하면 다음과 같이 더 정확하게 나눌 수 있습니다.

학생 / 명사

들 / 접미사

이 / 조사

공부 / 명사

한다 / 동사

즉, 문장 분석 단계에서는 문장을 형태소나 명사 단위로 분리합니다.

워드 클라우드에서는 보통 의미가 뚜렷한 **명사**를 많이 사용합니다.

예를 들어 다음 문장에서는

인공지능은 데이터를 학습한다

명사만 추출하면 다음과 같습니다.

인공지능

데이터

이렇게 분석해야 실제로 중요한 키워드를 찾을 수 있습니다.

4. 텍스트에서 많이 나온 단어 확인

마지막 단계는 분석된 단어들이 각각 몇 번 등장했는지 계산하는 과정입니다.

예를 들어 다음과 같은 단어 목록이 있다고 하겠습니다.

데이터, 인공지능, 데이터, 분석, 데이터, 모델

빈도를 계산하면 다음과 같습니다.

단어	빈도
데이터	3
인공지능	1
분석	1
모델	1

파이썬에서는 Counter 또는 pandas 의 value_counts()를 사용할 수 있습니다.

```

from collections import Counter

word_count = Counter(words)
print(word_count)

# 또는
import pandas as pd

word_count = pd.Series(words).value_counts()
print(word_count)

```

이 결과를 이용하면 어떤 단어가 가장 많이 등장했는지 알 수 있습니다.

워드 클라우드에서는 많이 등장한 단어일수록 크게 표시되고, 적게 등장한 단어일수록 작게 표시됩니다.

■ 워드 클라우드를 활용한 단어 빈도 시각화 실습해보기

❖ 정규 표현식을 만들어 불필요한 단어 제외하기

■ 대괄호 안에 패턴의 범위 지정

- `ㅏ-ㅣ` : 자음과 모음을 포함한 전체
- `가-힣` : 한글 문자 전체
- `ㅏ-ㅣ 가-힣` : 공백을 포함한 전체 한글 문자
- `^` : (캐럿 기호) 활용 시 **not** 의 의미로 반대가 됨
- `^ ㅏ-ㅣ 가-힣` : 한글과 공백 문자를 제외한 나머지 모든 문자
- `+` : 한 문자 이상
- `[^ ㅏ-ㅣ 가-힣]+` : 한글 및 공백을 제외한 1 글자 이상의 모든 문자 , 한글이나 공백이 아니면 모두 이 규칙(패턴)에 부합

■ 반복 처리

```
for i in range(len(line)):
```

- 라인의 길이만큼 반복

■ 정규표현식 치환

```
compile.sub()
```

- `compile` 변수에 저장된 정규식 규칙으로 문자열 치환

■ 문자열 정제

t = compile.sub(',', line[i])

- line 의 i 번째 문장에서
- 한글과 공백을 제외한 문자 제거
- 특수문자, 숫자, 영문 등을 삭제

■ 워드 클라우드 표현 실습해보기

❖ 실습 과정

1. 워드 클라우드 패키지 가져오기

2. 워드 클라우드 설치 및 불용어 사전 가져오기

- pip install wordcloud
 - 워드 클라우드 설치
- from wordcloud import WordCloud, STOPWORDS
 - 워드 클라우드 클래스와 불용어 사전 가져오기

3. 파일 다운로드 및 저장

- alice_novel.txt 파일 가져오기
- alice_novel = open('alice_novel.txt', 'r').read()
- - 파일 열기 및 내용 읽기

4. 불용어 사전 가져오기

- STOPWORDS 변수에 의미가 없는 불용어들이 들어가 있어 제거 필요

5. 워드 클라우드 생성하기

- max_words = 2000
- STOPWORDS 지정
- alice_wc.generate(alice_novel)
-

6. show 함수 호출하여 워드 클라우드 시각화하기

7. fig 함수 호출하여 워드 클라우드 너비와 높이 지정하기

- 너비 14
- 높이 18

으로 워드 클라우드 크기 지정